МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет прикладной математики и информатики
Кафедра технологий программирования

ОТЧЕТ

по лабораторной работе № 4
по курсу «Проектирование человеко-машинных интерфейсов»

Стили интеграции программных систем и проектирование API

Вариант: подсистема записи на мероприятия университетских клубов (Campus Events), группа 13

Выполнил: студент 13 группы
Zeng Yaoxin

Проверила: ст. преподаватель
Давидовская М.И.

Минск, 2026

СОДЕРЖАНИЕ

1 Цель работы и состав выполненных работ 3
2 Объект проектирования и действующие лица 3
3 Задание 1. Модель требований и реестр вариантов использования 4
4 Задание 2. Логическая и физическая модели базы данных 6
5 Задание 3. Маршруты и конечные точки API 8
6 Задание 4. Спецификация OpenAPI и тестирование на Mock-сервере 9
7 Задание 5. MCP-сервер (выполнено дополнительно) 12
8 Организация работы над проектом и документирование 13
9 Выводы 14
Список использованных источников 14

1 Цель работы и состав выполненных работ

Цель работы — получить навыки проектирования и документирования API; проектирования, документирования и интеграции агентных систем на основе протокола MCP; создания окружения на основе технологии контейнеризации Docker.

Объектом проектирования является подсистема записи на мероприятия университетских клубов (Campus Events). Модель требований и архитектура системы разработаны в лабораторной работе № 3, поэтому настоящая работа опирается на её результаты: требования уточнены под API, спроектированы логическая и физическая модели базы данных, маршруты и конечные точки, создана спецификация OpenAPI 3.0.3 и выполнено её тестирование на Mock-сервере.

Согласно критериям оценивания, для группы 13 обязательными являются задания 1–4; задание 5 (MCP-сервер) выполнено дополнительно. Работа индивидуальная: все материалы размещены в открытом репозитории GitHub, продублированы в вики и на сайте GitHub Pages.

Таблица 1 — Состав выполненных работ

Задание	Результат	Файлы в репозитории	Страница вики
1. Модель требований и реестр вариантов использования	20 вариантов использования, 4 действующих лица, диаграмма Use case, текстовые сценарии	docs/02-requirements.md	Функциональные требования
2. Логическая и физическая модели базы данных	14 таблиц, 13 перечислений, DDL, ER-диаграмма, 15 проверок ограничений	db/schema.sql, db/verify.sql	Схема базы данных
3. Маршруты и конечные точки API	25 маршрутов, 29 операций, 28 строк соответствия «вариант использования — HTTP-метод»	api/openapi.yaml	API проекта
4. Спецификация и тестирование API	OpenAPI 3.0.3 (28 схем), Mock-сервер, 8 сценариев проверки, снимки ответов в Bruno	api/, api/mock/	API проекта
5. MCP-сервер (дополнительно)	3 инструмента, 2 ресурса, 1 шаблон промпта, проверка в MCP Inspector	mcp/	MCP-сервер

2 Объект проектирования и действующие лица

Система автоматизирует запись студентов и сотрудников на мероприятия, которые проводят университетские клубы: хакатоны, лекции, спортивные турниры, экскурсии. Основные сущности — клуб, мероприятие,

сессия (конкретные дата, время и место проведения), регистрация, билет с QR-кодом, отзыв.

Выделены четыре действующих лица. Гость просматривает афишу и карточки мероприятий без авторизации. Участник записывается на сессию, отменяет запись, показывает билет и оставляет отзыв. Организатор создаёт мероприятия и сессии, видит список участников и выгружает его. Администратор модерирует публикации, управляет клубами и назначает роли. Вход выполняется через университетский CAS: тикет обменивается на JWT, дальнейшие запросы авторизуются заголовком Authorization.

Ключевые ограничения предметной области, определившие проектные решения: количество мест на сессии ограничено, и переполнение невозможно ни при параллельных запросах, ни при отменах; использованный билет нельзя погасить повторно; принятое решение по заявке на модерацию не изменяется.

3 Задание 1. Модель требований и реестр вариантов использования

Требования лабораторной работы № 3 дополнены до уровня, достаточного для проектирования API, и описаны трёхуровневой моделью «Действующее лицо → User Story → Use Case». Реестр содержит 20 вариантов использования (UC-01 … UC-23), сгруппированных по функциональным областям: афиша и поиск, запись и билеты, работа организатора, модерация и администрирование, уведомления.

Диаграмма вариантов использования (рисунок 1) построена в PlantUML и содержит четыре актора и пять смысловых разделов: регистрация, профиль, организация мероприятий, модерация и аналитика, каталог и поиск. Исходный код диаграммы хранится в репозитории рядом со скриптом сборки, поэтому изображение пересобирается одной командой при изменении требований.

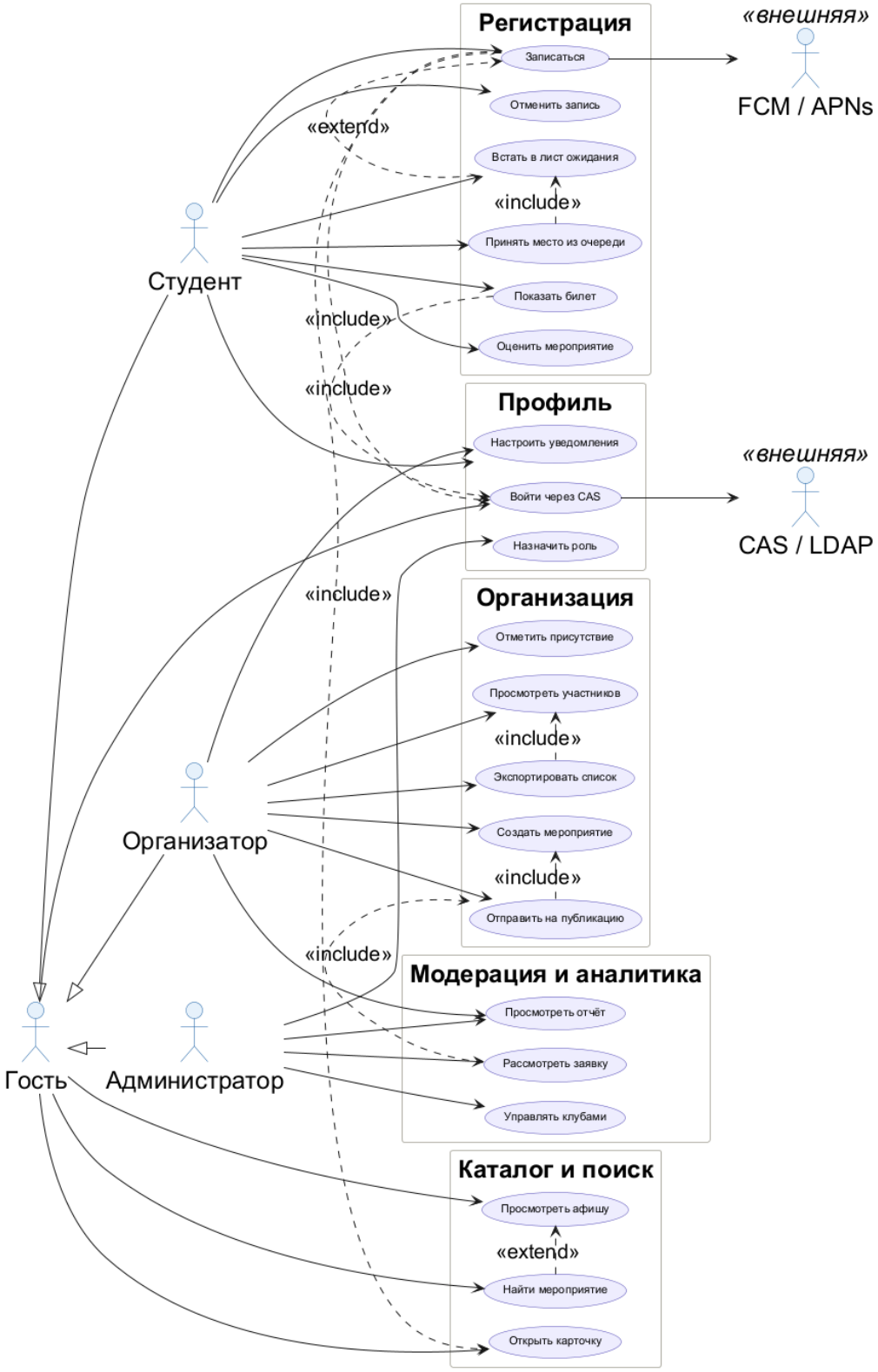


Рисунок 1 — Диаграмма вариантов использования: подсистема записи на мероприятия

Для каждого варианта использования в реестре указан основной поток — конечные точки API, которые его реализуют, и исключительные ситуации с кодами ошибок. Фрагмент реестра приведён в таблице 2; полный реестр

размещён на странице «Функциональные требования» в вики проекта и на сайте GitHub Pages.

Таблица 2 — Реестр вариантов использования (фрагмент)

Вариант использования	Основной поток	Исключительные ситуации
UC-01 Просмотреть афишу	GET /events с фильтрами и постраничной выдачей; карточки с датой, клубом и числом свободных мест	пустая выдача → 200, items: [] и подсказка снять фильтры
UC-06 Отменить запись	DELETE /registrations/{id}; место возвращается в фонд сессии, билет гасится	билет уже отмечен → 409 TICKET_USED; чужая запись → 403 FORBIDDEN
UC-09 Показать билет	GET /tickets, GET /tickets/{id}; QR-код рисуется на клиенте из qr_payload	билет отозван → 409 TICKET_USED
UC-11 Создать мероприятие	POST /events → черновик с сессиями и вместимостью	название короче 3 символов → 422 TITLE_TOO_SHORT; сессия в прошлом → 422 SESSION_IN_PAST
UC-12 Отправить на публикацию	POST /events/{id}/submit; создаётся версия карточки и заявка на модерацию	нет ни одной сессии → 422; заявка уже на рассмотрении → 409
UC-15 Экспортировать список участников	GET /sessions/{sid}/export → CSV или XLSX	пустой список → 200 с файлом, содержащим только заголовок
UC-23 Назначить роль	PATCH /users/{id}/role; изменение попадает в журнал	отзыв последнего администратора → 409; нет прав → 403 ROLE_REQUIRED

Текстовые сценарии оформлены для вариантов UC-05, UC-07/08, UC-13 и UC-17 — тех, где логика нетривиальна (лист ожидания, пакетная отметка присутствия, модерация); для каждого указаны предусловия, основной поток, альтернативы и постусловия. Остальные 15 вариантов описаны таблицей «шаг — действие — результат»: для линейных сценариев подробная форма избыточна.

4 Задание 2. Логическая и физическая модели базы данных

Логическая модель уточнена: для каждой из 14 сущностей указаны первичный ключ, внешние ключи и кардинальность связей (таблица 3). Преобладают связи «один ко многим»: клуб → мероприятия, мероприятие → сессии, сессия → регистрации, регистрация → билет. Связь «многие ко многим» между пользователем и клубом разрешена через таблицу membership, которая хранит роль и дату вступления.

Таблица 3 — Сущности базы данных

Таблица	Назначение
club	Клуб: объединение организаторов, проводит мероприятия
category	Справочник тематик для афиши и фильтров
app_user	Учётная запись: роль, контакты, признак активности
event	Мероприятие: карточка в афише, статус публикации
session	Сессия: дата, время, место, вместимость; единица записи
membership	Членство пользователя в клубе с ролью
registration	Регистрация пользователя на сессию
ticket	Билет: QR-код и отметка присутствия
seat_offer	Предложение места из листа ожидания
event_version	Снимок карточки мероприятия для модерации
moderation_request	Заявка на модерацию: очередь проверки
decision	Решение модератора по заявке, неизменяемое
review	Отзыв: оценка 1–5 и текст после посещения
notification	Уведомление: письмо или напоминание о событии

Физическая модель реализована для PostgreSQL 16: 14 таблиц, 13 перечислений, 103 поля, три триггера и семь частичных уникальных индексов. Первичные ключи — UUID, время хранится в timestamptz в UTC, значения перечислений совпадают с подписями интерфейса, чтобы не поддерживать два словаря.

Целостность обеспечена средствами СУБД, а не кодом приложения: триггеры на таблице registration пересчитывают занятые места при вставке и обновлении и не позволяют записать больше вместимости, триггер trg_decision_no_update запрещает изменять уже принятое решение. Частичные уникальные индексы не дают одному пользователю дважды записаться на одну сессию и дважды оставить отзыв о мероприятии.

DDL и демонстрационные данные (34 строки) загружаются автоматически при старте контейнера PostgreSQL, поэтому развёртывание схемы сводится к одной команде. Для проверки ограничений написан сценарий db/verify.sql из 15 атакующих запросов: он выполняется на реальной СУБД и завершается откатом транзакции. Все 15 проверок пройдены — переполнение сессии, повторная запись, двойная отметка билета, изменение решения и нарушение диапазона оценки блокируются базой.

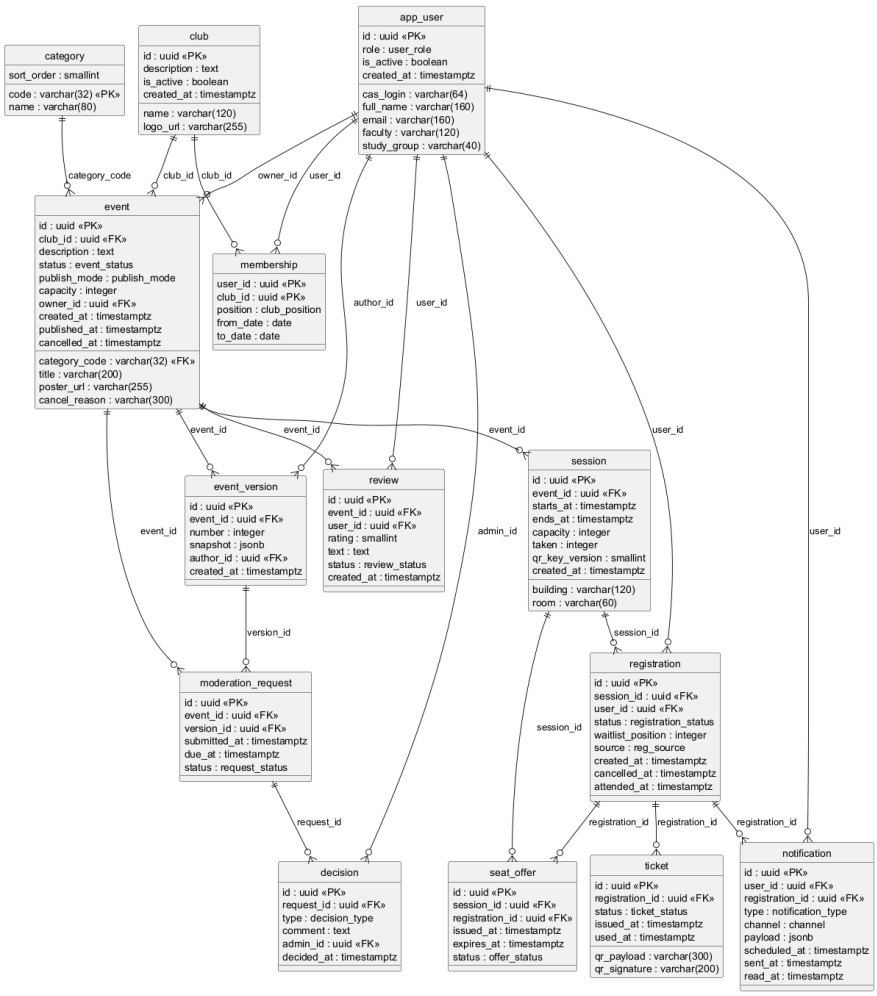


Рисунок 2 — ER-диаграмма физической модели базы данных

5 Задание 3. Маршруты и конечные точки API

Спроектировано 25 маршрутов и 29 операций. Коллекции ресурсов именуются во множественном числе, вложенность не превышает двух уровней, действия, не сводимые к CRUD, оформлены как подресурсы: POST /events/{id}/submit, POST /sessions/{sid}/waitlist, POST /tickets/{id}/check-in. Полный перечень приведён в таблице 4.

Таблица 4 — Маршруты и конечные точки API

Метод	Маршрут	Назначение
GET	/events	Афиша с фильтрами и постраничной выдачей
POST	/events	Создать мероприятие (черновик)
GET	/events/{id}	Карточка мероприятия с сессиями
PATCH	/events/{id}	Изменить мероприятие
POST	/events/{id}/submit	Отправить мероприятие на публикацию
GET	/events/{id}/report	Сводка по мероприятию
GET	/search	Поиск по названию и описанию
GET	/categories	Справочник категорий
GET	/clubs	Список клубов

Метод	Маршрут	Назначение
POST	/clubs	Создать клуб
PATCH	/clubs/{id}	Изменить или закрыть клуб
POST	/auth/cas/callback	Обмен тикета CAS на токен доступа
GET	/me	Текущий пользователь, его роль и клубы
GET	/notifications/settings	Настройки каналов уведомлений
PUT	/notifications/settings	Изменить каналы уведомлений
PATCH	/users/{id}/role	Назначить роль пользователю
POST	/sessions/{sid}/registrations	Записаться на сессию
DELETE	/registrations/{id}	Отменить запись
POST	/sessions/{sid}/waitlist	Встать в лист ожидания
POST	/seat-offers/{id}/accept	Принять предложенное место
GET	/tickets	Мои билеты
GET	/tickets/{id}	Билет с данными для QR-кода
POST	/tickets/{id}/check-in	Отметить присутствие по билету
POST	/checkins/batch	Пакетная отметка после работы без сети
GET	/sessions/{sid}/participants	Список участников сессии
GET	/sessions/{sid}/export	Выгрузка списка участников
POST	/reviews	Оставить отзыв о посещённом мероприятии
GET	/moderation/requests	Очередь заявок на модерацию
POST	/moderation/requests/{id}/decision	Принять решение по заявке

Каждый вариант использования сопоставлен с HTTP-методом и маршрутом — всего 28 строк соответствия, они опубликованы на странице «API проекта» в вики. Такой реестр позволяет проверить в обе стороны: ни одно требование не осталось без конечной точки, и у каждой конечной точки есть основание в требованиях.

Единый формат ошибки — объект из четырёх полей: code (машиночитаемый код, например NO_SEATS или TICKET_USED), message (текст для пользователя на русском языке), field (поле, не прошедшее проверку, или null) и traceId (идентификатор запроса для поиска в журналах). Коды ответов используются по назначению: 201 при создании, 204 при удалении, 401 при отсутствии или истечении токена, 403 при недостатке прав, 409 при конфликте состояния, 422 при ошибке валидации, 429 при превышении лимита запросов.

6 Задание 4. Спецификация OpenAPI и тестирование на Mock-сервере

Спецификация оформлена в формате OpenAPI 3.0.3: 28 схем компонентов, описание безопасности (Bearer JWT), общие ответы и параметры пагинации. Для каждого POST- и PUT-запроса определены тело,

обязательные поля, ограничения и полный набор статусов с пояснением смысла. Спецификация проходит проверку валидатором openapi-spec-validator без ошибок и предупреждений.

Тестирование выполнено на Mock-сервере Prism 5, развёрнутом локально. Prism отдаёт ответ, описанный в контракте, а конкретный код запрашивается заголовком Prefer: code=<код>, поэтому проверка не зависит от случайной генерации данных и воспроизводится у любого проверяющего. Прогон из восьми сценариев (таблица 5) выполняется скриптом api/mock/smoke.py; результат — 8 из 8 PASS, полный протокол с телами запросов и ответов сохранён в репозитории.

Таблица 5 — Сценарии проверки на Mock-сервере

Сценарий	Запрос	Код	Итог
Афиша: успешный список	GET /events?size=2&page=1	200	PASS
Создание мероприятия: черновик принят	POST /events	201	PASS
Запись на сессию: место есть	POST /sessions/{sid}/registrations	201	PASS
Запись на сессию: свободных мест нет	POST /sessions/{sid}/registrations	409	PASS
Запись без токена	POST /sessions/{sid}/registrations	401	PASS
Список заявок модерации: чужая организация	GET /moderation/requests	403	PASS
Создание мероприятия: поле не прошло проверку	POST /events	422	PASS
Афиша: превышен лимит запросов	GET /events	429	PASS

Дополнительно контракт проверен в графическом клиенте Bruno: импортирована коллекция из восьми запросов, три ответа приведены на рисунках 3–5. Ответ 201 подтверждает приём черновика мероприятия, ответ 409 NO_SEATS — что на сессии не осталось свободных мест, ответ 403 — что очередь модерации недоступна пользователю без нужной роли.

Побочный результат прогона: Mock-сервер выявил дефект контракта. Общий компонент ответа 422 возвращал пример «комментарий короче 10 символов» для всех пятнадцати операций, включая создание мероприятия. Пример перенесён в операцию решения по заявке, а общий компонент теперь отдаёт VALIDATION_FAILED с полем title. Это показывает, что проверка на Mock-сервере работает не только как демонстрация, но и как ревью контракта.

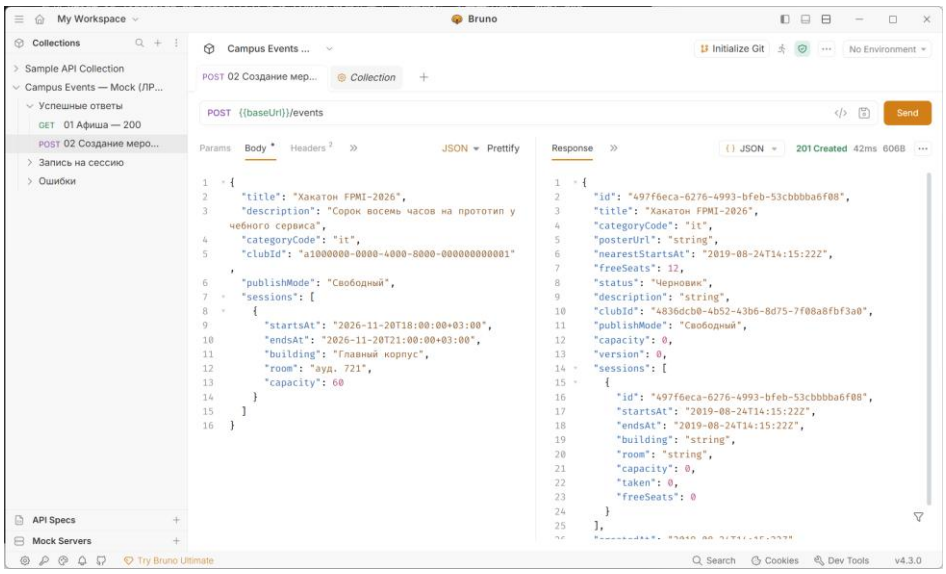


Рисунок 3 — Проверка POST /events: ответ 201 Created

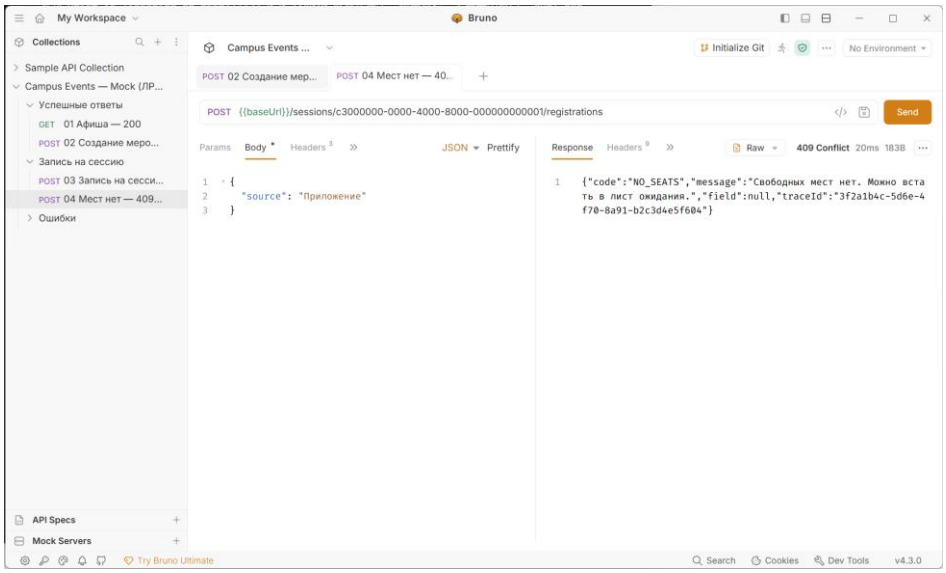


Рисунок 4 — Проверка записи на сессию: ответ 409, свободных мест нет

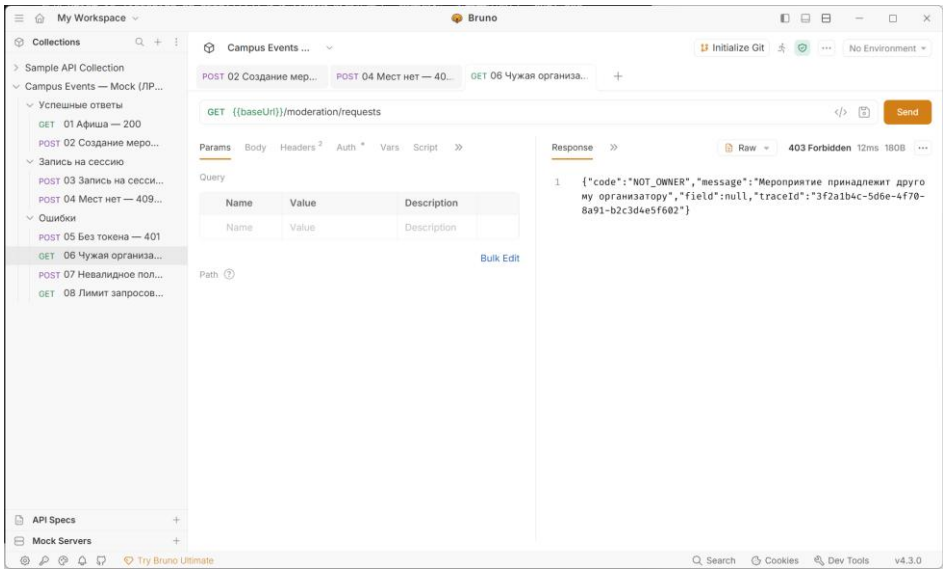


Рисунок 5 — Проверка очереди модерации: ответ 403, недостаточно прав

7 Задание 5. MCP-сервер (выполнено дополнительно)

Для группы 13 задание 5 не является обязательным; MCP-сервер реализован как MVP на MCP Python SDK и демонстрирует, как AI-ассистент работает с подсистемой через три инструмента: create_registration (запись на сессию), cancel_registration (отмена записи) и check_in_ticket (отметка присутствия по QR-коду).

Данные ассистент читает через два ресурса: catalog://events — каталог ближайших мероприятий со свободными местами, и шаблонный ресурс registrations://user/{user_id} — записи конкретного пользователя. Для типового сценария подготовлен шаблон промпта registration_help, задающий порядок действий: прочитать каталог, уточнить сессию у пользователя, вызвать инструмент.

Главный принцип — правила проверяет сервер, а не модель: аргументы инструментов валидируются по JSON-схеме, вместимость считает база данных, повторная отметка билета идемпотентна и возвращает already_marked вместо ошибки, отмена использованного билета запрещена.

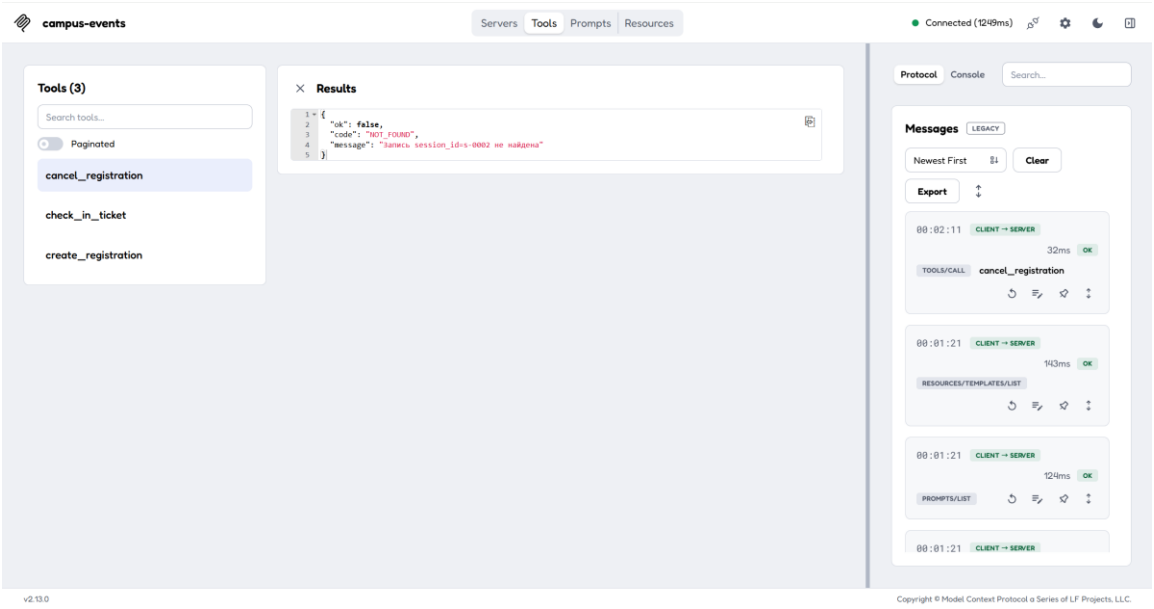


Рисунок 6 — MCP Inspector: инструменты сервера и результат вызова

Сервер проверен официальным MCP Inspector (рисунок 6): получены списки инструментов, ресурсов и промптов, выполнены вызовы create_registration, check_in_ticket (дважды, для проверки идемпотентности) и

вызов с несуществующей сессией, который вернул NOT_FOUND. Протокол проверки в режиме командной строки сохранён в репозитории, дополнительно 10 сценариев покрыты скриптом test_mcp.py. Инструкция по развёртыванию и примеры запросов от лица пользователя оформлены в mcp/README.md и на странице «MCP-сервер» в вики.

8 Организация работы над проектом и документирование

Проект ведётся в GitHub Projects: четыре колонки To do / In progress / To check / Done, одна задача — один issue с критерием приёмки. На рисунке 7 показано состояние доски на момент сдачи. Работа индивидуальная, поэтому коммиты идут в ветку main небольшими шагами «одна задача — один коммит»; для доработок создаются отдельные ветки.

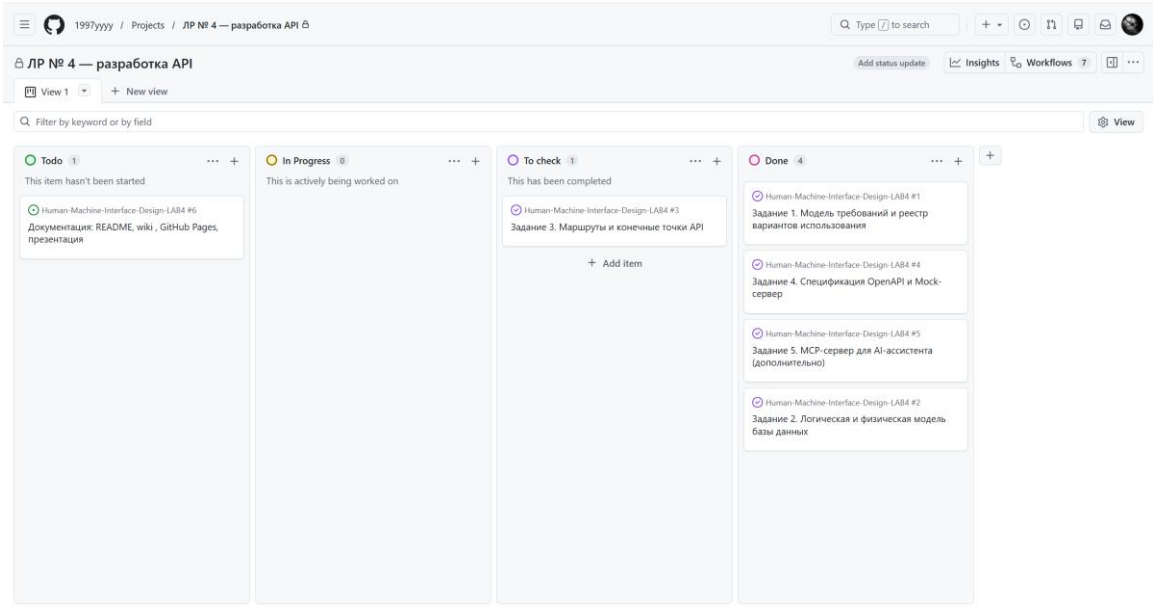


Рисунок 7 — Доска проекта в GitHub Projects

Документация построена по требованиям методических указаний. README содержит разделы Project Name, Description, Installation, Sub modules, Usage и Contributing. Вики состоит из восьми страниц — семи обязательных и страницы MCP-сервера. Сайт GitHub Pages повторяет структуру вики; дополнительно на нём опубликованы читаемые HTML-версии спецификации API и DDL, поскольку GitHub отдаёт файлы .yaml

и .sql без указания кодировки, из-за чего кириллица в них отображается некорректно.

Репозиторий проекта: https://github.com/1997yyyy/Human-Machine-Interface-Design-LAB4. Сайт проекта: https://1997yyyy.github.io/Human-Machine-Interface-Design-LAB4/

9 Выводы

1. Требования лабораторной работы № 3 доведены до уровня, пригодного для проектирования API: 20 вариантов использования, четыре действующих лица, диаграмма Use case и текстовые сценарии.

2. Спроектированы логическая и физическая модели данных. Целостность обеспечена средствами PostgreSQL, а не кодом приложения, и подтверждена 15 проверками на реальной СУБД.

3. Спроектированы 25 маршрутов и 29 операций REST API; каждый вариант использования сопоставлен с конечной точкой, определён единый формат ошибок.

4. Создана и проверена спецификация OpenAPI 3.0.3. Тестирование на Mock-сервере (8 из 8) выявило дефект контракта, который был устранён.

5. Реализован MCP-сервер с тремя инструментами, двумя ресурсами и шаблоном промпта; работа проверена официальным MCP Inspector.

6. Освоены GitHub Projects, вики и GitHub Pages как средства ведения и документирования проекта.

Список использованных источников

1. Давидовская М.И. Проектирование человеко-машинных интерфейсов: метод. указания к лаб. работе № 4. — Минск: БГУ, 2026. — 16 с.
2. Проектирование REST API на примере интернет-магазина // GetAnalyst.
3. Пример проектирования REST API библиотеки // GetAnalyst.
4. Материалы по REST API, Postman и Swagger // GetAnalyst.
5. OpenAPI Specification 3.0.3: https://spec.openapis.org/oas/v3.0.3
6. SwaggerHub Documentation: https://support.smartbear.com/swaggerhub/docs/
7. Bruno Documentation. Mock Servers: https://docs.usebruno.com/
8. Model Context Protocol: https://modelcontextprotocol.io/
9. GitHub Docs: boards, wikis, Pages: https://docs.github.com/